

AI3604 Computer Vision

Kailing Wang 521030910356

September 28.2023

Written Assignment

- a. If the circle is allowed to lie in a plane that is tilted with respect to the image plane, we know the projection can be a circle, an ellipse or a line. However, here we restrict it to be on a plane parallel to the image plane, the projection is obviously still a circle.

Figure 1. shows the case (and also shows this homework is not finished by GPT, maybe). I use the same coordinate as slides. Suppose we have a circular disk described by:

$$\begin{cases} (x - x_0)^2 + (y - y_0)^2 = r^2 \\ z = z_0 \end{cases}, \quad (1)$$

where $\mathbf{O}_0(x_0, y_0, z_0)$ is the center of the disk and r is the radius. For a point $\mathbf{P}(x_1, y_1, z_1)$ on the disk, we use similar triangles to find the projected point $\mathbf{P}'(x'_1, y'_1, z'_1)$:

$$\frac{x'_1}{f} = \frac{x_1}{z_1}, \frac{y'_1}{f} = \frac{y_1}{z_1}, \quad (2)$$

here f is the focal length. $\mathbf{P}$ is on disk:

$$\left(\frac{x'_1 z_1}{f} - x_0\right)^2 + \left(\frac{y'_1 z_1}{f} - y_0\right)^2 = r^2, \quad (3)$$

The projection on image plane can be described by:

$$\begin{cases} (x'_1 - \frac{x_0 f}{z_1})^2 + (y'_1 - \frac{y_0 f}{z_1})^2 = \frac{r^2 f^2}{z_1^2} \\ z = f \end{cases}, \quad (4)$$

and it is a circle.

- b. The normal vector of the plane is $(A, B, C)^T$. Thus for the first case where $A = C = D = 0, B = 1$, the normal vector is $\mathbf{n} = (0, 1, 0)^T$. The plane, let's call it $\mathcal{F}_1$, is perpendicular to the y-axis at $y = 0$, as shown in Figure 2. We represent a line with:

$$\begin{cases} x = x_0 + t \cdot d_x \\ y = y_0 + t \cdot d_y \\ z = z_0 + t \cdot d_z \end{cases}, \quad \text{or } \mathbf{x} = \mathbf{x}_0 + t\mathbf{d}. \quad (5)$$

For lines on $\mathcal{F}_1$, we have $\mathbf{d} \cdot \mathbf{n} = 0$, which means $d_y = 0$ while d_x and d_z are arbitrary. In class we know the vanishing point of the line is $(f \frac{d_x}{d_z}, f \frac{d_y}{d_z})$. This shows the vanishing points are on line.

$$\begin{cases} y = 0 \\ z = f \end{cases}. \quad (6)$$

Therefore, for three given directions $(1, 0, 2)$, $(2, 0, 1)$ and $(1, 0, 1)$ on the plane, their vanishing point are $(\frac{f}{2}, 0)$, $(2f, 0)$, $(f, 0)$ respectively.

Similarly, for the second case, the vanishing points are on

$$\begin{cases} x = 0 \\ z = f \end{cases}. \quad (7)$$

For three given directions $(0, 1, 2)$, $(0, 2, 1)$ and $(0, 1, 1)$ on the plane, their vanishing point are $(0, \frac{f}{2})$, $(0, 2f)$, $(0, f)$ respectively.

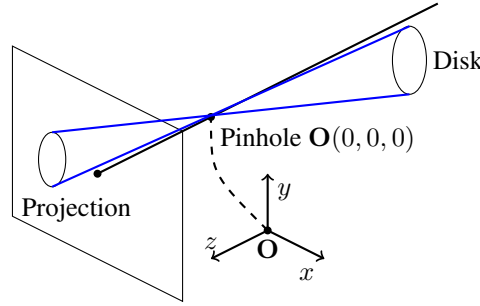


Figure 1: Disk before pinhole camera



Figure 2: Vanishing point of planes

- c. In the last problem we have $\mathbf{d} \cdot \mathbf{n} = 0$, or $d_x \cdot A + d_y \cdot B + d_z \cdot C = 0$. We can further get

$$f \frac{d_x}{d_z} \cdot A + f \frac{d_y}{d_z} \cdot B + fC = 0, \quad (8)$$

and this shows $x \cdot A + y \cdot B + f \cdot C = 0$ is the vanishing points of all lines on the given plane.

Programming Assignment

- 1) a. To do this, since opencv returns a numpy array, simply use `np.where` to replace values higher than threshold with 255 else 0. It's silly to use if-else and scan instead. The choice of threshold is highly dependent to the image. Neither mean, medium nor diagram valleys would be proper. For given pictures, 128 works fine.

- b. I use a union-find set to store labels. In my implementation, I scan from the bottom-left corner as clarified(in slides we start from top-left as default, however).

I first initialize the first row of pixels. Then I scan the image with a 2×2 array $\begin{bmatrix} A & B \\ C & D \end{bmatrix}$. Before scanning each row, the first column pixel A is initialized by C and D . During scanning, we merge labels of A and D using union-find set and update B if B is not background. B will be assigned the label of A, C, D (sequence do not matter) or a new label if A, C, D are background.

In the end, I paint the labels with gray-scale color. To better visualize, I used a gamma transform function with parameter 2.2.

- c. I implement this function completely based on equations on slides. Nothing special here.

The images are shown in Figure 3. The first row are binary and the second are labeled. The output of three images are(the same sequence as images, kept the first two labels each with .2f):

```
{'position': {'x': 349.33, 'y': 215.45}, 'orientation': -1.26, 'roundness': 1.30}
{'position': {'x': 195.32, 'y': 222.38}, 'orientation': 0.69, 'roundness': 2.04}

{'position': {'x': 303.57, 'y': 177.27}, 'orientation': 0.41, 'roundness': 2.42}
{'position': {'x': 417.72, 'y': 240.29}, 'orientation': -0.78, 'roundness': 40.93}

{'position': {'x': 413.66, 'y': 203.95}, 'orientation': -1.12, 'roundness': 3.94}
{'position': {'x': 265.97, 'y': 168.65}, 'orientation': -0.49, 'roundness': 1.75}
```

- 2) a. The given `coins.png` has obvious edges, and thus smaller Sobel operator(i.e. 3×3) should work fine. To compare, I referred to the official doc of OpenCV. No significant difference is found.

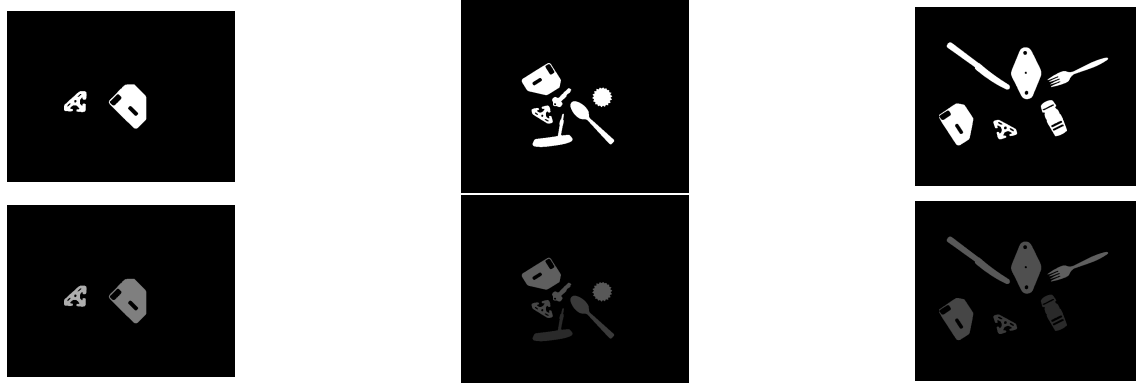


Figure 3: Output Images

- b. I used `np.where` to finish the first task of this problem. For the second one, I considered traverse all the theta from 0 to 360 degree. At first I doubted it, for some pixels are counted multiple times. However, I get to know that people indeed do the same thing while rasterizing. In fact, multiple counts implies the pixel is closer to the round edge than others.

It really takes a second using merely `np`. I checked the array from problem 1. The maximum reaches 781, and thus I choose 350 as threshold. Since the circles are obvious enough, I did not further modify. Choices of radius are from 15 to 35. To calculate faster, I calculate the whole image as one for each theta. Furthermore, the coins do not look like standard circles, so I added ellipse correction.

- c. This function is only to find the local maximum and draw circles. Nothing is special. The threshold is to make every coin circled. Some coins are circled multiply times, and this is because they are not standard circles. Pick anyone of the output circle is fine.

The output are in Figure 7. The program finishes within 2 seconds on an old CPU.

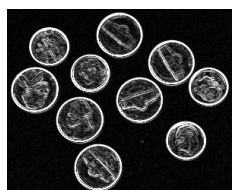


Figure 4: edge_raw

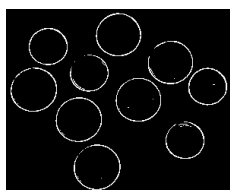


Figure 5: coins_edge

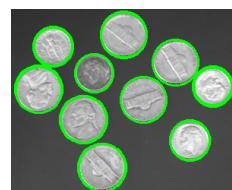


Figure 6: coins_circle

Figure 7: Output Images 2

